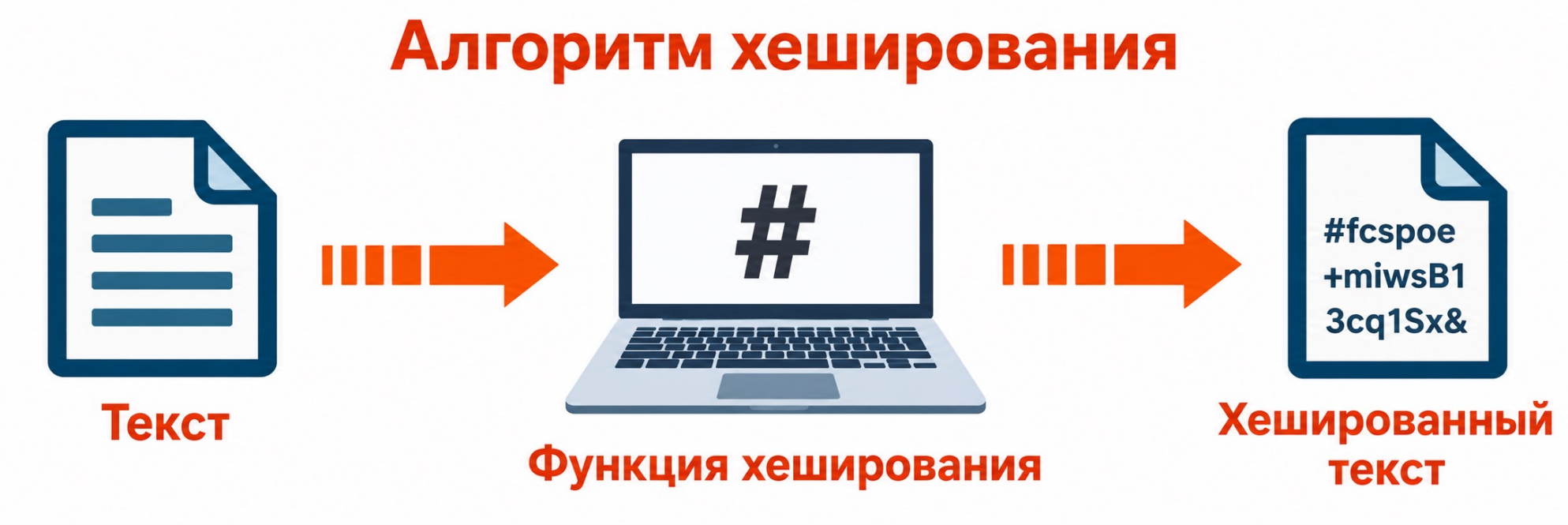
Криптографические хеш-функции

- **Хеширование** – это преобразование массива входных данных произвольной длины в битовую строку фиксированной длины с помощью специальной функции.



Определение 8.2. *Хеш-функцией* называется любая функция

$$y = h(x_1 x_2 \dots x_n),$$

которая строке (сообщению) $x_1 x_2 \dots x_n$ *произвольной* длины n ставит в соответствие целое число *фиксированной* длины.

Определение 8.2. *Хеш-функцией* называется любая функция

$$y = h(x_1 x_2 \ldots x_n),$$

которая строке (сообщению) $x_1 x_2 \ldots x_n$ *произвольной* длины n ставит в соответствие целое число *фиксированной* длины.

Примером хеш-функции может служить контрольная сумма для сообщения. В этом случае

$$h(x_1 x_2 \ldots x_n) = (x_1 + x_2 + \ldots + x_n) \bmod 2^w,$$

где w — размер машинного слова. Длина слова, получаемого как

Хеш-функции можно разделить на три основных класса:

- криптографические хеш-функции
- контрольные суммы
- хеш-функции, использующиеся для ускорения поиска данных и построения хеш-таблиц

Криптографические хеш-функции

На такие функции накладываются особые требования надёжности. К ним относятся функции семейств:

MD (MD2, MD4, MD5, MD6),

SHA (SHA-1, SHA-2, SHA-3),

ГОСТ (ГОСТ Р 34.11-94, ГОСТ Р 34.11-2012)

и другие.

Контрольные суммы

Это быстрые и простые алгоритмы, находящие применение, как правило, в проверке целостности принятых данных.

Контрольные суммы

Это быстрые и простые алгоритмы, находящие применение, как правило, в проверке целостности принятых данных.

Такие хеш-функции помогают обнаружить непреднамеренные ошибки в сообщениях, однако не спасают от преднамеренных ошибок (легко внести изменения в сообщение так, чтобы контрольная сумма осталась прежней).

Контрольные суммы

Это быстрые и простые алгоритмы, находящие применение, как правило, в проверке целостности принятых данных.

Такие хеш-функции помогают обнаружить непреднамеренные ошибки в сообщениях, однако не спасают от преднамеренных ошибок (легко внести изменения в сообщение так, чтобы контрольная сумма осталась прежней).

Самый известный представитель – **CRC-32**.

К третьему классу относятся **хеш-функции, использующиеся для ускорения поиска данных и построения хеш-таблиц**.

К третьему классу относятся **хеш-функции, использующиеся для ускорения поиска данных и построения хеш-таблиц**.

Хеш-функции, относящиеся ко второму и третьему классу, не подходят для криптографических алгоритмов, так как не удовлетворяют базовым требованиям, предъявляемым к криптографическим хеш-функциям.

Особенностями криптографических хеш-функций являются:

1. **Необратимость**, которая заключается в следующем:

для известного m вычислительно неосуществимо найти блок данных X, для которого H(X) = m. Если хеш-функция обладает данным свойством, ее также называют односторонней. Это требование всегда накладывается на хеш-функции, которыми шифруются пароли для безопасного хранения в системе.

Особенностями криптографических хеш-функций являются:

1. **Необратимость**, которая заключается в следующем:

для известного m вычислительно неосуществимо найти блок данных X, для которого H(X) = m. Если хеш-функция обладает данным свойством, ее также называют односторонней. Это требование всегда накладывается на хеш-функции, которыми шифруются пароли для безопасного хранения в системе.

2. Стойкость к коллизиям 1 рода. При таком условии очень трудоемко (практически неосуществимо) для известного сообщения M подобрать другое сообщение N, для которого H(M) = H(N).

3. Стойкость к коллизиям 2 рода. Она предполагает, что почти невозможно (очень сложно вычислительно) подобрать пару сообщений M, M’, имеющих совпадающие хеш-значения.

3. Стойкость к коллизиям 2 рода. Она предполагает, что почти невозможно (очень сложно вычислительно) подобрать пару сообщений M, M’, имеющих совпадающие хеш-значения.

4. Лавинный эффект.

Он заключается в том, что при малейшем изменении аргумента значение функции сильно меняется.

3. Стойкость к коллизиям 2 рода. Она предполагает, что почти невозможно (очень сложно вычислительно) подобрать пару сообщений M, M’, имеющих совпадающие хеш-значения.

4. Лавинный эффект.

Он заключается в том, что при малейшем изменении аргумента значение функции сильно меняется.

5. Скорость вычисления.

Для любого заданного x вычисление h(x) должно выполняться относительно быстро.

Коллизия - ситуация, когда двум различным сообщениям соответствуют одинаковые значения хеш-функции.

Поскольку хеш-функции являются сжимающими (отображают множество в менее мощное множество), то коллизии неизбежны, и становится важна вычислительная сложность их нахождения.

Коллизия - ситуация, когда двум различным сообщениям соответствуют одинаковые значения хеш-функции.

Поскольку хеш-функции являются сжимающими (отображают множество в менее мощное множество), то коллизии неизбежны, и становится важна вычислительная сложность их нахождения.

Отличие коллизий **первого** и **второго рода**:

в первом случае дан конкретный исходный блок, цель - подобрать такой другой исходный блок с таким же хеш-значением (чтобы произошла коллизия);

во втором случае конкретного исходного сообщения нет, и нужно просто найти любую пару с одинаковыми хеш-значениями.

Пример.

Пусть предполагаемая хеш-функция описана следующим образом: каждую букву в слове заменяем на её порядковый номер и полученные числа суммируем.

Пример.

Пусть предполагаемая хеш-функция описана следующим образом: каждую букву в слове заменяем на её порядковый номер и полученные числа суммируем.

Множество исходных блоков - множество существующих осмысленных в русском языке слов. В этом случае задача подбора другого слова с тем же хешем довольно сложная (подберите, например, коллизию к слову "криптография").

Пример.

Пусть предполагаемая хеш-функция описана следующим образом: каждую букву в слове заменяем на её порядковый номер и полученные числа суммируем.

Множество исходных блоков - множество существующих осмысленных в русском языке слов. В этом случае задача подбора другого слова с тем же хешем довольно сложная (подберите, например, коллизию к слову "криптография").

А вот к коллизиям второго рода эта функция однозначно не стойка: достаточно взять любую анаграмму, например: "кот" и "ток".

Можно увидеть, что функция, нестойкая к коллизиям первого рода, нестойка к коллизиям второго рода.

Обратное же неверно, так как устойчивость к коллизиям второго рода более сильная, чем к коллизиям первого рода (если функция устойчива к коллизиям второго рода, она автоматически устойчива и к коллизиям первого рода).

В **схеме электронной подписи** могут использоваться криптографические хеш-функции, устойчивые к коллизиям **первого рода**.

Процесс хеширования является необратимым, что позволяет обезопасить данные от изменения злоумышленниками.

Если даже оригинальный документ или транзакция будут в дальнейшем изменены, то в результате они получат другую цифровую подпись, что сигнализирует о несоответствии в системе.

Создание криптографических хеш-функций, удовлетворяющих вышеописанным свойствам, представляет собой сложную задачу.

Самой первой конструкцией для создания таких хеш-функций является схема **Меркля-Дамгарда**, предложенная Р. Мерклем и И. Дамгардом в 1979 году.

В ее основе лежит «функция сжатия», обычно являющаяся необратимой, например, блоковый шифр или его аналог.

Конструкция Меркля-Дамгарда легла в основу таких хеш-функций, как **MD 2, MD4, MD 5, MD 6**, линейка **RIPEMD**, **ГОСТ Р 34.11-94**, **Стрибог (ГОСТ Р 34.11-2012)**, а также поколений функций **SHA-0, SHA-1, SHA-2**.

Конструкция Меркля-Дамгарда легла в основу таких хеш-функций, как **MD 2, MD4, MD 5, MD 6**, линейка **RIPEMD**, **ГОСТ Р 34.11-94**, **Стрибог (ГОСТ Р 34.11-2012)**, а также поколений функций **SHA-0, SHA-1, SHA-2**.

Описание этих и подобных им функций довольно сложное и громоздкое. Мы рассмотрим универсальный способ построения хеш-функций на базе блоковых шифров, который представляет практический интерес, хотя получаемые хеш-функции и не являются очень быстро вычислимыми.

Именно такой подход использован в российском стандарте на криптографическую хеш-функцию (**ГОСТ Р34.11-94**).

Пусть дан блоковый шифр E, который для заданного блока X и ключа K формирует шифротекст Y,

$$Y = E_K(X).$$

Мы представим два алгоритма, для которых длина слова, получаемого как значение хеш-функции, равна размеру блока в шифре, но отметим, что известны конструкции, позволяющие получать хеш-функции с длинами слов, кратными размеру блока.

В первом алгоритме сообщение вначале представляется в виде последовательности блоков $X_1, X_2, \ldots, X_n$. Последний блок при необходимости дополняется нулями, иногда в последний блок приписывают длину сообщения в виде двоичного числа. Значение хеш-функции h получается как результат выполнения следующего итерационного процесса:

$$
\begin{aligned}
&h \leftarrow 0;\\
&\text{FOR } i = 1, 2, \ldots, n \text{ DO}\\
&\qquad h \leftarrow E_h(X_i) \oplus X_i\,.
\end{aligned}
$$

В качестве начального значения h можно использовать не нуль, а какое-либо «магическое» число, но это не имеет большого значения. В данном алгоритме значение h, полученное на предыдущей итерации, используется в качестве ключа шифра в следующей итерации. Поэтому неявно полагается, что длина ключа в шифре равна длине блока. Однако, как мы видели при изучении шифра RC6, длина ключа может значительно превышать размер блока (в RC6 при максимальной длине блока 256 бит длина ключа может достигать 255 байт, или 2040 бит). В таких случаях более эффективен другой алгоритм.

В этом алгоритме сообщение вначале представляется в виде последовательности $X_1, X_2, \ldots, X_m$, в которой размер каждого элемента равен длине ключа в шифре. Последний элемент заполняется так же, как и в первом алгоритме. Значение хеш-функции h вычисляется следующим образом:

$$
\begin{aligned}
&h \leftarrow 0;\\
&\text{FOR } i = 1, 2, \ldots, m \text{ DO}\\
&\qquad h \leftarrow E_{X_i}(h) \oplus h.
\end{aligned}
$$

Здесь уже элементы сообщения выполняют роль ключей в шифре.

Конструкция Меркля-Дамгарда легла в основу таких хеш-функций, как:

- **MD 2, MD4, MD 5, MD 6**,
- линейка **RIPEMD**,
- **ГОСТ Р 34.11-94**,
- **Стрибог (ГОСТ Р 34.11-2012)**,
- а также поколений функций **SHA-0, SHA-1, SHA-2**.

После дискредитации линейки функций **MD** разработчики начали искать альтернативный подход к реализации хеш-функций. Таким подходом стала конструкция **«sponge»**, используемая в поколении **SHA-3**.

Как было сказано ранее, в алгоритме «слепой» подписи, на котором основывается система электронного анонимного голосования, используется хеш-функция SHA3-512 (одна из разновидностей SHA-3).

SHA-3 (Keccak) — алгоритм хеширования переменной разрядности, разработанный группой авторов во главе с Йоаном Дайменом.

Как было сказано ранее, в алгоритме «слепой» подписи, на котором основывается система электронного анонимного голосования, используется хеш-функция SHA3-512 (одна из разновидностей SHA-3).

SHA-3 (Keccak) — алгоритм хеширования переменной разрядности, разработанный группой авторов во главе с Йоаном Дайменом.

2 октября 2012 года Keccak стал победителем конкурса криптографических алгоритмов, проводимым Национальным институтом стандартов и технологий США, **5 августа 2015** года алгоритм утверждён и опубликован в качестве стандарта FIPS 202. В аппаратных реализациях Keccak оказался достаточно быстрым.

Хеш-функция **Keccak** с может иметь разную длину выхода – 224, 256, 384 или 512 бит. В основе алгоритма **Keccak** лежит конструкция под названием **Sponge** – «губка». Данная структура криптографических алгоритмов была предложена авторами алгоритма **Keccak** ранее, ее можно представить следующим образом:

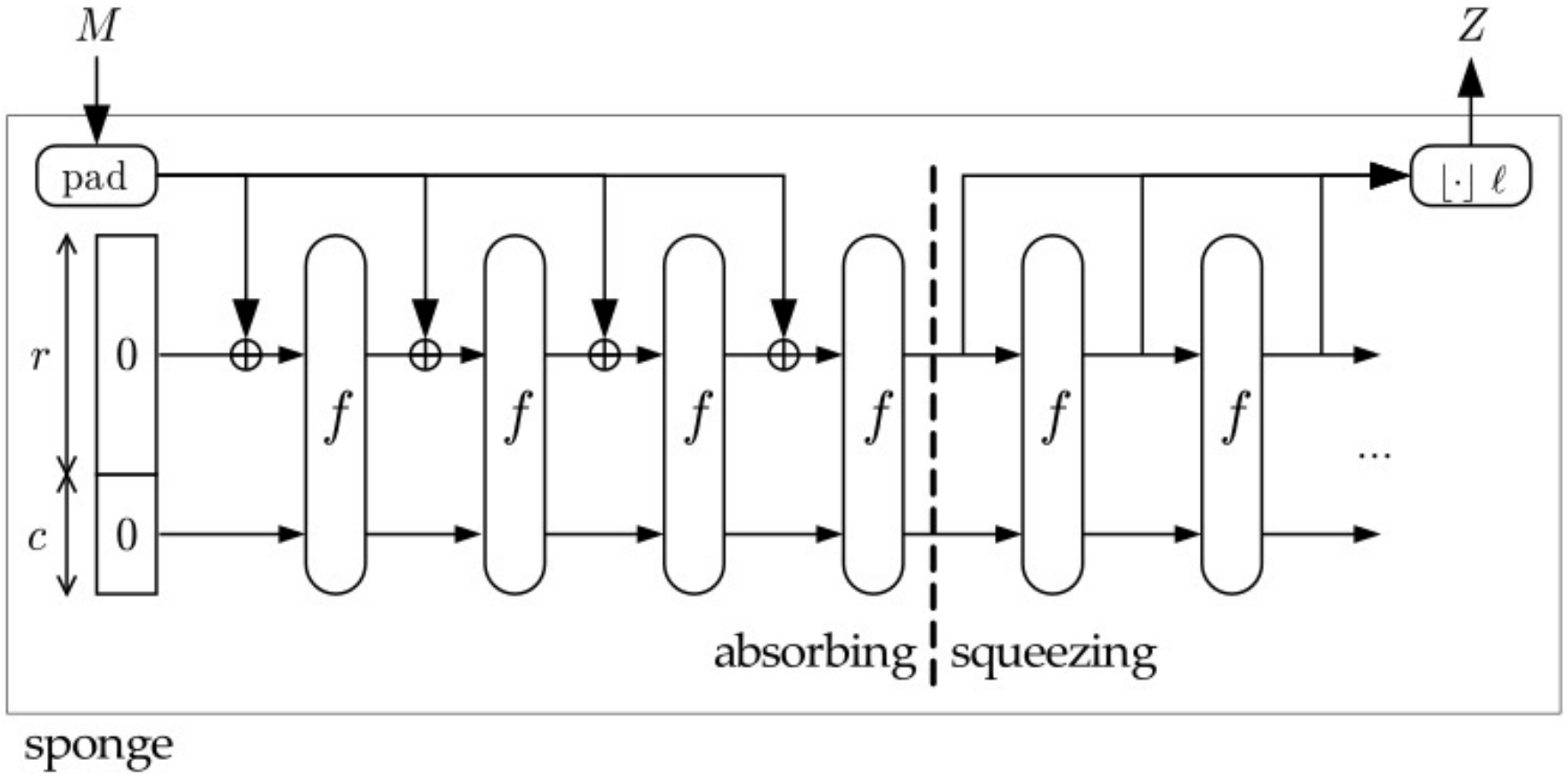


Схема состоит из двух этапов:

1. **Absorbing (впитывание).** Исходное сообщение M подвергается многораундовым перестановкам f.

2. **Squeezing (отжатие).** Вывод получившегося в результате перестановок значения Z.

Варьируя значение переменных **r** и **c**, можно получить абсолютно разные хеш-функции.

Для **SHA3-512** в качестве этих значений нужно выбирают **r = 576, c = 1024**.

Для получения хеш-значения нужно:

Взять исходное сообщение **M** и дополнить его до длины, кратной **r**:

M = M || 0x01 || 0x00 || ... || 0x00 || 0x80.

То есть к сообщению дописывается единичный байт, необходимое количество нулей и завершающий байт со значением **0x80**. Однако в случае, если необходимо дополнить всего один байт, достаточно добавить лишь **0x81**.

Затем для каждого блока M_i длиной r бит выполняется следующее:

1. Сложение по модулю 2 с первыми r битами набора начальных состояний S. Перед началом работы функции все элементы S будут равны нулю.

2. N раз применяем к полученным в результате данным функцию f. Набором начальных состояний S для блока M_{i+1} будет результат последнего раунда блока M_i.

3. После того, как все блоки M_i закончатся, берется итоговый результат и возвращается в качестве хеш-значения.

Хеш-функция ведёт себя как случайная функция:

- для случайно выбираемого *x* все значения $y = h(x)$ равновероятны
- даже если $x2$ зависит от $x1$, $h(x2)$ и $h(x1)$ выглядят как статистически независимые
- при изменении одного бита в x изменяется примерно половина бит в $h(x)$
- для произвольно выбираемого x вероятность того, что в $h(x)$ первые k бит будут нулевые, равна $1/2^k$

Представленные алгоритмы вычисления хеш-функций удовлетворяют всем требованиям, предъявляемым к криптографическим хеш-функциям, в предположении стойкости используемых блоковых шифров.

ЭЛЕКТРОННАЯ, ИЛИ ЦИФРОВАЯ ПОДПИСЬ

Прежде чем начать рассмотрение криптографической цифровой подписи, сформулируем три свойства, которым (в идеале) должна удовлетворять любая, в частности, обычная рукописная подпись:

1. Подписать документ может только «законный» владелец подписи (и, следовательно, никто не может подделать подпись).

2. Автор подписи не может от нее отказаться.

3. В случае возникновения спора возможно участие третьих лиц (например, суда) для установления подлинности подписи.

ЭЛЕКТРОННАЯ, ИЛИ ЦИФРОВАЯ ПОДПИСЬ



Абонент выбирает случайно два больших простых числа P и Q. Затем он вычисляет число

$$N = PQ. \tag{2.28}$$

(Число N является открытой информацией, доступной другим абонентам.) После этого абонент вычисляет число $\phi = (P-1)(Q-1)$ и выбирает некоторое число $d < \phi$, взаимно простое с ϕ, и по обобщенному алгоритму Евклида находит число c, такое, что

$$cd \bmod \phi = 1. \tag{2.29}$$

Пусть Алиса хочет подписать сообщение $\bar{m} = m_1, \ldots, m_n$. Тогда вначале она вычисляет так называемую хеш-функцию

$$y = h(m_1, \ldots, m_n),$$

которая ставит в соответствие сообщению $\bar{m}$ число y. Предполагается, что алгоритм вычисления хеш-функции всем известен.

Алиса вычисляет число

$$s = y^c \bmod N, \tag{4.1}$$

т.е. она возводит число y в свою секретную степень. Число s это и есть цифровая подпись. Она просто добавляется к сообщению $\bar{m}$, и тем самым Алиса имеет сформированное подписанное сообщение

$$\langle \bar{m},\ s \rangle . \tag{4.2}$$

Теперь каждый, кто знает открытые параметры Алисы, ассоциированные с ее именем, т.е. числа N и d, может проверить подлинность ее подписи. Для этого необходимо, взяв подписанное сообщение (4.2), вычислить значение хеш-функции $h(\bar{m})$, число

$$w = s^d \bmod N \tag{4.3}$$

и проверить выполнение равенства $w = h(\bar{m})$.

Утверждение 4.1. *Если подпись подлинная, то* $w = h(\bar{m})$.

Доказательство. Из (4.3), (4.1) и свойств схемы RSA (см. разд. 2.6) следует

$$w = s^d \bmod N = y^{cd} \bmod N = y = h(\bar{m}).$$

Утверждение 4.2. *Описанная электронная подпись удовлетворяет всем требованиям, предъявляемым к подписи.*

Пример 4.1. Пусть $P = 5$, $Q = 11$. Тогда $N = 5 \cdot 11 = 55$, $\phi = 4 \cdot 10 = 40$. Пусть $d = 3$. Такой выбор d возможен, так как $\gcd(40, 3) = 1$. Параметр $c = 3^{-1} \bmod 40$ вычисляем с помощью обобщенного алгоритма Евклида (см. разд. 2.3), $c = 27$.

Утверждение 4.2. *Описанная электронная подпись удовлетворяет всем требованиям, предъявляемым к подписи.*

Пример 4.1. Пусть $P = 5$, $Q = 11$. Тогда $N = 5 \cdot 11 = 55$, $\phi = 4 \cdot 10 = 40$. Пусть $d = 3$. Такой выбор d возможен, так как $\gcd(40, 3) = 1$. Параметр $c = 3^{-1} \bmod 40$ вычисляем с помощью обобщенного алгоритма Евклида (см. разд. 2.3), $c = 27$.

Пусть, например, Алиса хочет подписать сообщение $\bar{m} = abbbaa$, для которого значение хеш-функции равно, скажем, 13:

$$y = h(abbbaa) = 13.$$

В этом случае Алиса вычисляет по (4.1)

$$s = 13^{27} \bmod 55 = 7$$

и формирует подписанное сообщение

$$\langle abbbaa,\ 7 \rangle .$$

Теперь тот, кто знает открытые ключи Алисы $N = 55$ и $d = 3$, может проверить подлинность подписи. Получив подписанное сообщение, он заново вычисляет значение хеш-функции

$$h(abbbaa) = 13$$

(если содержание сообщения не изменено, то значение хеш-функции совпадет с тем, которое вычисляла Алиса) и вычисляет по (4.3)

$$w = 7^3 \bmod 55 = 13.$$

Значения w и хеш-функции совпали, значит, подпись верна. $\square$